

Projet « Mon Miam Miam »

Documentation des diagrammes

Module Expérience Étudiant & Commandes

Auteure Mukendi Kanyiki Wivine

Rôle Conception des API RESTful

Version V1.0

Table des matières

1 Introduction **2**
1.1 Les diagrammes du module . . . 2
1.2 Outils . . . 2
1.3 Périmètre . . . 2

2 Conventions de lecture **2**

3 Diagramme de cas d'utilisation **3**
3.1 Objectif . . . 3
3.2 Acteurs . . . 3
3.3 Cas d'utilisation . . . 3
3.4 Relations entre cas . . . 3

4 Diagramme de classes **6**
4.1 Objectif . . . 6
4.2 Classes . . . 6
4.3 Relations . . . 6
4.4 Valeurs fixes (énumérations) . . . 7
4.5 Règles de gestion portées par le modèle . . . 7

5 Diagrammes de séquence **9**
5.1 Inscription d'un étudiant . . . 9
5.2 Connexion d'un étudiant . . . 9
5.3 Validation d'une commande avec points de fidélité . . . 12

6 Lien avec les spécifications **13**

7 Points à valider avec le groupe **13**

1 Introduction

Ce document accompagne le dossier de spécifications des API du module **Expérience Étudiant & Commandes** de l'application « Mon Miam Miam » (restaurant ZeDuc@Space). Il documente les cinq diagrammes réalisés pour ce module : ce que chacun représente, comment le lire, et à quelle partie des spécifications il se rattache.

1.1 Les diagrammes du module

N°	Diagramme	Type UML	Fichier image
1	Cas d'utilisation du module	Cas d'utilisation	`diagramme_cas_utilisation.png`
2	Classes du module	Classes	`diagramme_classes.png`
3	Inscription d'un étudiant	Séquence	`sequence_inscription.png`
4	Connexion d'un étudiant	Séquence	`sequence_connexion.png`
5	Validation d'une commande avec points de fidélité	Séquence	`sequence_commande.png`

1.2 Outils

Le diagramme de cas d'utilisation et le diagramme de classes sont réalisés avec draw.io. Les trois diagrammes de séquence sont écrits en PlantUML et exportés en image. Leurs sources (fichiers `.drawio` et `.puml`) sont conservées avec le projet.

1.3 Périmètre

Ce document ne couvre que le module de l'étudiant. Les diagrammes des modules Fidélité, Parrainage & Animation et Back-Office & Administration sont documentés par leurs auteures respectives.

2 Conventions de lecture

— **Cas d'utilisation :** un acteur est représenté par un personnage, un cas d'utilisation par une ellipse. Le cadre délimite le module. Un trait plein relie l'acteur au cas qu'il déclenche.

— **Relation *include* :** le cas de départ utilise toujours le cas d'arrivée (flèche pointillée `«include»`).

— **Relation *extend* :** le cas de départ ajoute un comportement facultatif au cas d'arrivée (flèche pointillée `«extend»`).

— **Classes :** chaque classe est un rectangle divisé en trois zones (nom, attributs, méthodes). Le signe `+` indique un élément public, `[0..1]` un champ facultatif.

— **Cardinalités :** `1` exactement un, `0..1` zéro ou un, `0..*` zéro ou plusieurs, `1..*` au moins un. Un losange plein marque une composition (la ligne de commande n'existe pas sans sa commande).

— **Énumérations :** les rectangles marqués `«enumeration»` listent les valeurs fixes autorisées pour un attribut.

— **Séquence :** les participants sont en haut, le temps descend. Une flèche pleine est un appel, une flèche pointillée une réponse. Un bloc `alt` sépare le déroulement normal du cas d'erreur.

3 Diagramme de cas d'utilisation

3.1 Objectif

Montrer ce que chaque acteur peut faire dans le module, avant d'entrer dans le détail technique. Chaque cas d'utilisation correspond à un ou plusieurs endpoints des spécifications.

3.2 Acteurs

Acteur	Type	Rôle
Visiteur	Principal	Personne non connectée : peut s'inscrire, se connecter, récupérer son mot de passe et consulter le menu du jour.
Étudiant	Principal	Étudiant connecté. Il hérite de toutes les actions du visiteur et peut en plus gérer son profil, commander, consulter son historique, commenter une commande livrée et gérer ses réclamations.
Service d'envoi d'email	Secondaire (système)	Envoie le lien de récupération du mot de passe.

3.3 Cas d'utilisation

N°	Cas d'utilisation	Acteur	Endpoint(s) associé(s)
1	S'inscrire	Visiteur	`POST /api/auth/register`
2	Se connecter	Visiteur	`POST /api/auth/login`
3	Récupérer son mot de passe	Visiteur	`POST /api/auth/forgot-password` `POST /api/auth/reset-password`
4	Consulter le menu du jour	Visiteur	`GET /api/menu/today`
5	Se déconnecter	Étudiant	`POST /api/auth/logout`
6	Consulter et modifier son profil	Étudiant	`GET /api/auth/me` `PATCH /api/auth/me`
7	Passer une commande	Étudiant	`POST /api/orders`
8	Choisir le mode de service (sur place ou livraison)	Étudiant	Champs `mode_service`, `heure_arrivee`, `adresse_livraison` de `POST /api/orders`
9	Utiliser des points de fidélité	Étudiant	Champ `points_a_utiliser` de `POST /api/orders`
10	Consulter l'historique des commandes	Étudiant	`GET /api/orders/history`
11	Voir le détail d'une commande	Étudiant	`GET /api/orders/{id}`
12	Laisser un commentaire après livraison	Étudiant	`POST /api/orders/{id}/review`
13	Déposer une réclamation	Étudiant	`POST /api/claims`
14	Suivre une réclamation	Étudiant	`GET /api/claims` `GET /api/claims/{id}`

3.4 Relations entre cas

— **Include :** « Passer une commande » *inclut* « Choisir le mode de service » : toute commande indique si elle est servie sur place (avec heure d'arrivée) ou livrée (avec adresse).

— **Extend :** « Utiliser des points de fidélité » *étend* « Passer une commande » : l'étudiant peut commander sans points.

— **Extend :** « Voir le détail d'une commande » *étend* « Consulter l'historique » : on y accède depuis la liste.

— **Extend :** « Laisser un commentaire » *étend* « Voir le détail d'une commande » : disponible uniquement si la commande est livrée.

— **Héritage :** l'étudiant *est un* visiteur connecté et reprend ses cas d'utilisation.

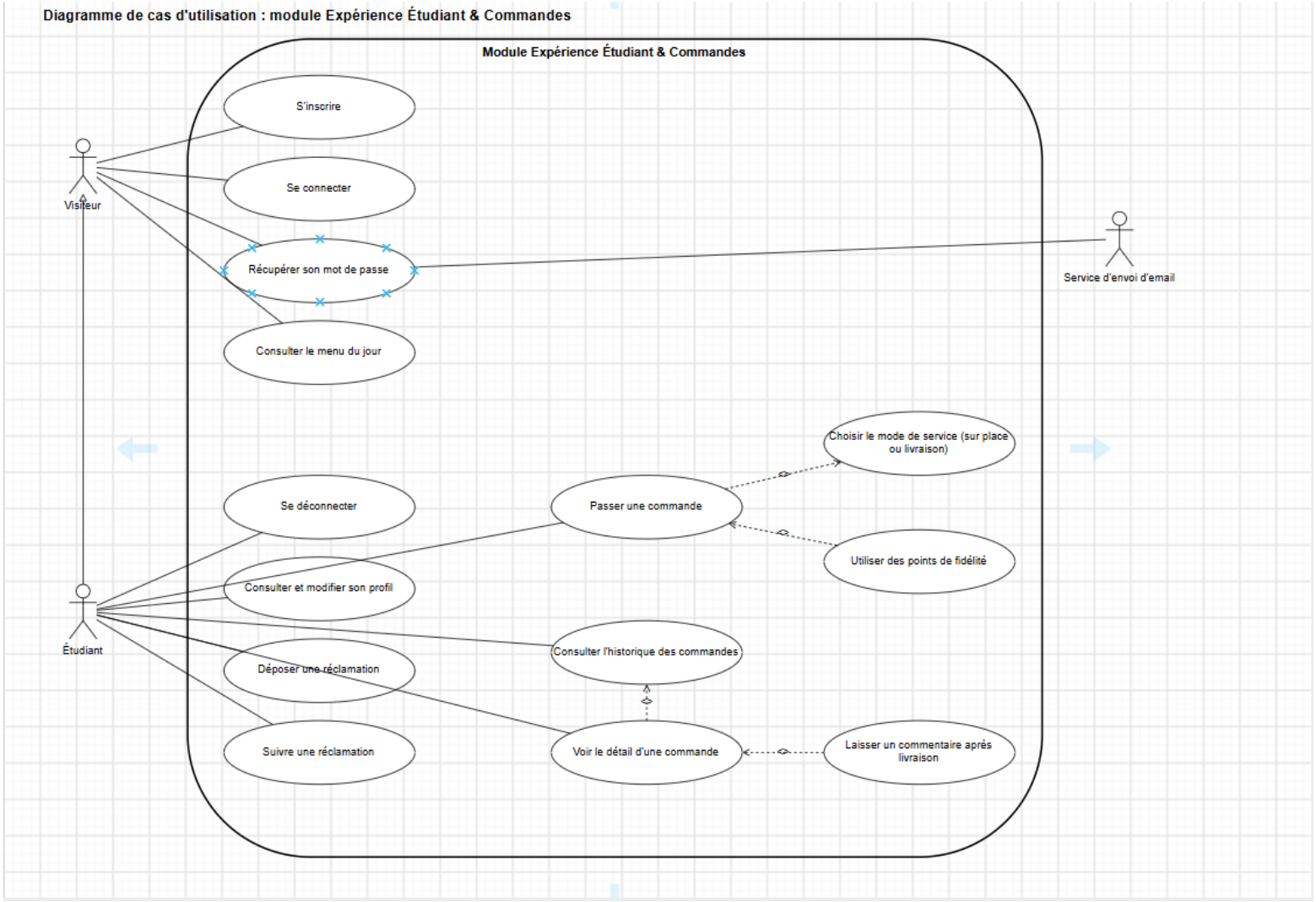


FIGURE 1 – Diagramme de cas d'utilisation du module Expérience Étudiant & Commandes

4 Diagramme de classes

4.1 Objectif

Décrire les données que le module manipule et la façon dont elles sont liées. Les noms d'attributs sont les mêmes que les champs des réponses JSON des API, ce qui évite toute conversion entre le frontend, le backend et la base de données.

4.2 Classes

Classe	Rôle	Attributs principaux
Utilisateur	Compte de l'étudiant (ou d'un autre rôle).	`nom`, `email`, `role`, `statut_compte`, `points_fidelite`, `date_inscription`
Produit	Plat ou boisson du menu.	`categorie`, `nom`, `prix`, `disponible`, `image_url`
Commande	Commande passée par un étudiant, avec le détail du calcul.	`statut`, `mode_service`, `sous_total`, `points_utilises`, `reduction_points`, `montant_net_a_payer`, `points_gagnes`
LigneCommande	Un produit commandé en une certaine quantité.	`quantite`, `prix_unitaire`, `total_ligne`
MouvementPoints	Gain, utilisation, expiration ou ajustement de points.	`type`, `points`, `solde_apres`, `commande_id`
Reclamation	Problème signalé sur une commande, avec la réponse du restaurant.	`objet`, `message`, `statut`, `reponse_message`, `traite_par`, `valide_par`
Avis	Commentaire laissé après livraison.	`note`, `commentaire`, `date_avis`

4.3 Relations

Relation	Cardinalités	Signification
Utilisateur *passe* Commande	1 — 0..*	Un étudiant peut passer plusieurs commandes.
Utilisateur *soumet* Reclamation	1 — 0..*	Un étudiant peut déposer plusieurs réclamations.
Utilisateur *possède* MouvementPoints	1 — 0..*	Chaque mouvement de points appartient à un étudiant.
Commande *contient* LigneCommande	1 — 1..* (composition)	Une commande a au moins une ligne ; sans commande, la ligne n'existe pas.
Produit *est commandé dans* LigneCommande	1 — 0..*	Un produit peut figurer dans plusieurs lignes.
Commande *fait l'objet de* Reclamation	1 — 0..*	Une réclamation concerne une seule commande.
Commande *reçoit* Avis	1 — 0..1	Au plus un commentaire par commande.
Commande *génère* MouvementPoints	0..1 — 0..*	Une commande crée des mouvements de points ; un mouvement peut ne pas avoir de commande (expiration, ajustement).

4.4 Valeurs fixes (énumérations)

Énumération	Valeurs	Utilisée par
RoleUtilisateur	etudiant, employe, gerant, administrateur	Utilisateur.role
StatutCompte	actif, inactif	Utilisateur.statut_compte
ModeService	sur_place, livraison	Commande.mode_service
StatutCommande	en_attente, en_preparation, prete, en_livraison, livree, annulee	Commande.statut
StatutPaiement	en_attente, paye, echoue	Commande.statut_paiement
StatutReclamation	ouverte, en_traitement, resolue, rejetee	Reclamation.statut
TypeMouvement	gain, utilisation, expiration, ajustement	MouvementPoints.type

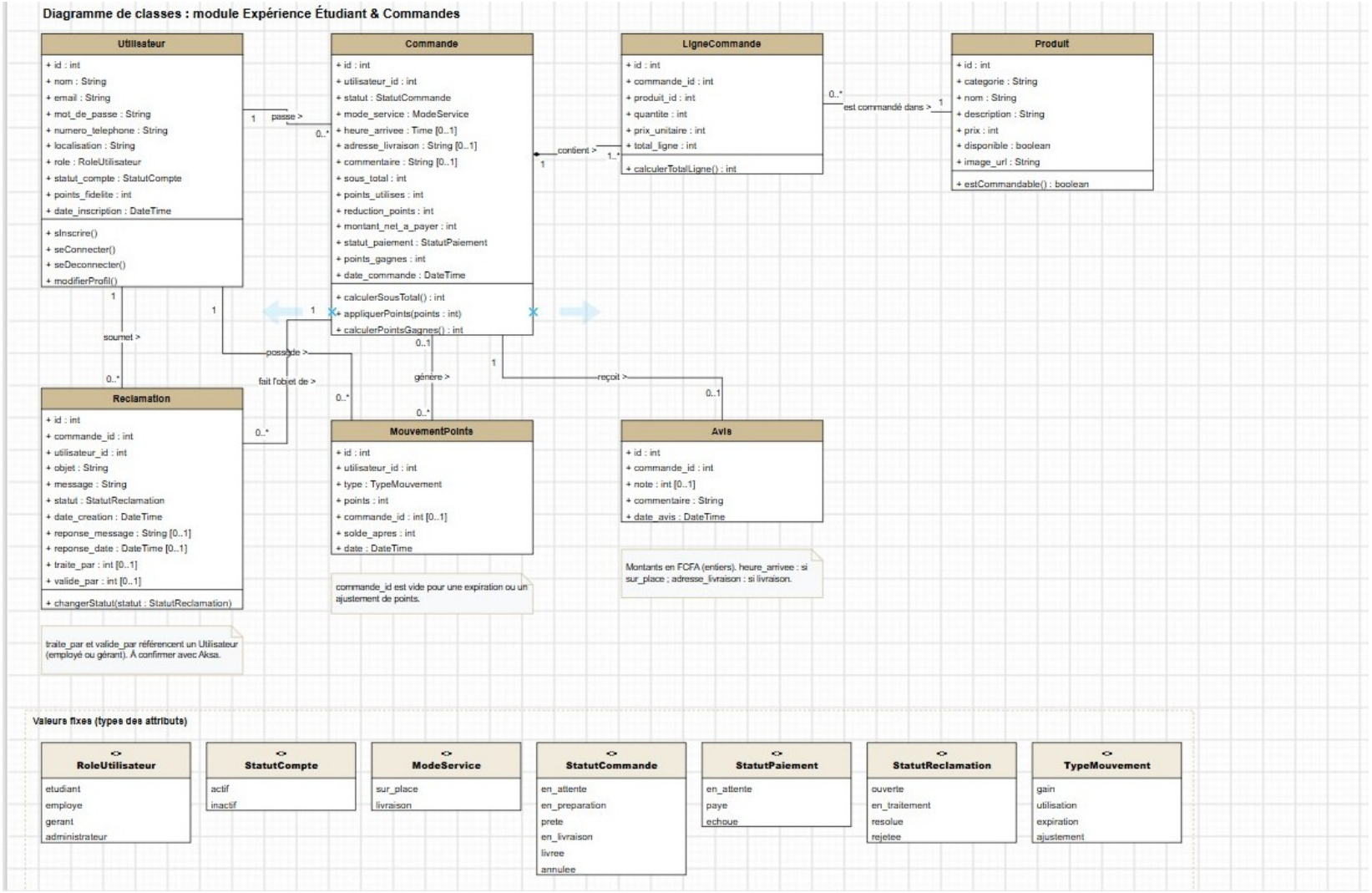
4.5 Règles de gestion portées par le modèle

— Les montants sont des entiers en FCFA, sans décimale.

— Le prix d'un produit est recopié dans `LigneCommande.prix_unitaire` au moment de la commande : un changement de prix ultérieur ne modifie pas les commandes passées.

— `heure_arrivee` n'est renseignée que pour une commande sur place ; `adresse_livraison` uniquement pour une livraison.

— Chaque mouvement de points enregistre le solde après l'opération (`solde_apres`), ce qui permet de reconstituer l'historique.



class.jpg

FIGURE 2 – Diagramme de cas d'utilisation du module Expérience Étudiant & Commandes

5 Diagrammes de séquence

Les trois diagrammes suivants montrent les échanges entre quatre participants : l'**étudiant**, le **frontend** (interface), l'**API Laravel** et la **base de données**.

5.1 Inscription d'un étudiant

Endpoint : `POST /api/auth/register`.

Déroulement normal

1. L'étudiant remplit le formulaire d'inscription (nom, email, mot de passe et sa confirmation, téléphone, localisation).
2. Le frontend envoie la requête à l'API.
3. L'API valide les champs : email valide et unique, mot de passe de 8 caractères minimum avec une majuscule et un chiffre, confirmation identique.
4. L'API crée l'utilisateur dans la base, avec le mot de passe haché.
5. La base confirme la création.
6. L'API crée le jeton d'accès (Laravel Sanctum).
7. L'API répond `201 Created` avec les données du compte et le jeton.
8. Le frontend donne accès à l'espace étudiant.

Cas d'erreur

— Données invalides, email déjà utilisé ou mot de passe non conforme : `422` avec le code `VALIDATION_ERROR` et le détail par champ ; le frontend affiche les erreurs sous chaque champ.

— Trop de requêtes : `429`.

5.2 Connexion d'un étudiant

Endpoint : `POST /api/auth/login`.

Déroulement normal

1. L'étudiant saisit son email et son mot de passe.
2. Le frontend envoie la requête à l'API.
3. L'API recherche l'utilisateur dans la base.
4. La base renvoie l'utilisateur et le mot de passe haché.
5. L'API vérifie le mot de passe (bcrypt) et le statut du compte.
6. L'API crée le jeton Sanctum.
7. L'API répond `200 OK` avec le jeton et le rôle.
8. Le frontend redirige l'utilisateur vers l'espace correspondant à son rôle.

Cas d'erreur

— Email ou mot de passe incorrect : `401`, `INVALID_CREDENTIALS`. Le message est volontairement le même dans les deux cas, pour ne pas révéler quels emails ont un compte.

— Compte désactivé : `403`, `ACCOUNT_DISABLED`.

— Plus de 5 tentatives par minute : `429`.

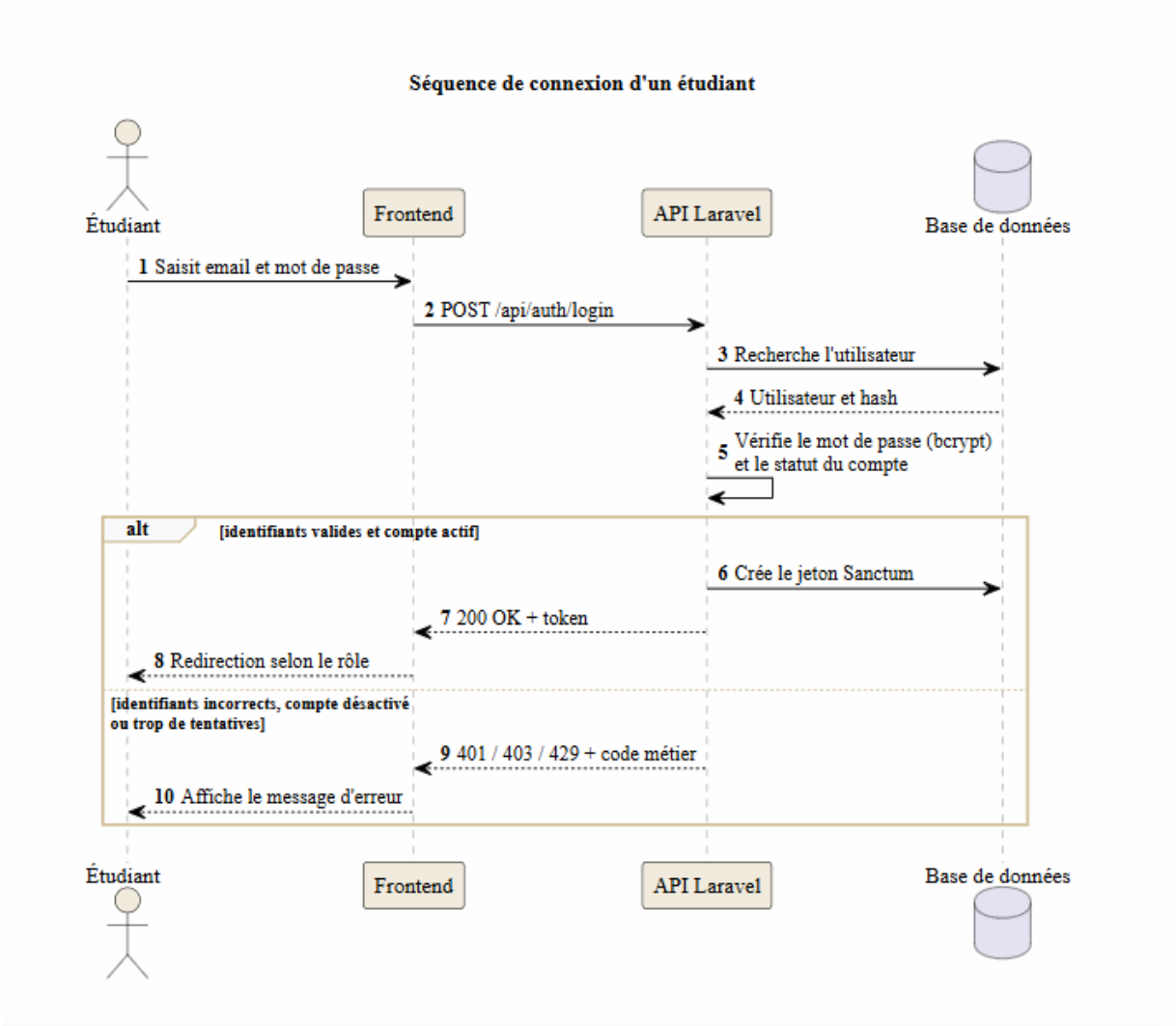


FIGURE 3 – Diagramme de séquence de connexion du module Expérience Étudiant & Commandes

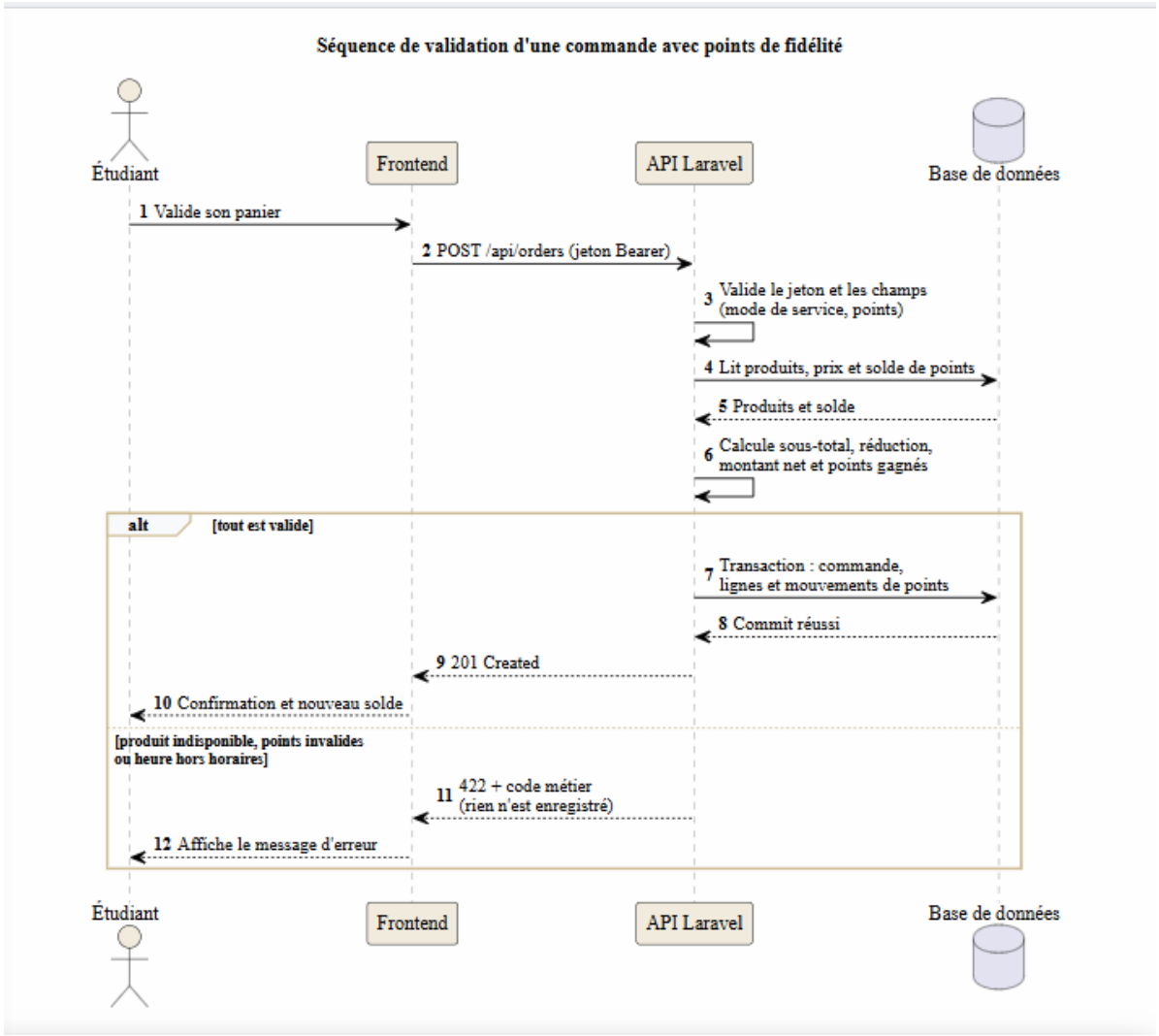


FIGURE 4 – Diagramme de séquence de commande du module Expérience Étudiant & Commandes

5.3 Validation d'une commande avec points de fidélité

Endpoint : `POST /api/orders`.

Déroulement normal

1. L'étudiant valide son panier.
2. Le frontend envoie la requête avec le jeton Bearer, les articles, le mode de service et les points à utiliser. Il n'envoie aucun prix.
3. L'API vérifie le jeton et valide les champs.
4. L'API lit en base les produits, leurs prix et le solde de points de l'étudiant.
5. La base renvoie ces données.
6. L'API calcule le sous-total, la réduction (15 points = 1 000 FCFA), le montant net et les points gagnés ($\lfloor$montant net/1000$\rfloor$).
7. L'API enregistre, dans une seule transaction, la commande, ses lignes et les mouvements de points.
8. La base confirme la transaction.
9. L'API répond `201 Created` avec le détail de la commande et le nouveau solde.
10. Le frontend affiche la confirmation et le nouveau solde.

Exemple chiffré (commande n°204)

Deux poulets à 3 500 FCFA et un jus à 1 000 FCFA donnent un sous-total de 8 000 FCFA. Avec 15 points utilisés, la réduction est de 1 000 FCFA, le montant net de 7 000 FCFA et les points gagnés sont 7. Le solde passe de 30 à $30 - 15 + 7 = 22$ points.

Cas d'erreur

Si une règle n'est pas respectée, l'API répond `422` avec un code métier, et **rien n'est enregistré** (la transaction est annulée) :

— `PRODUIT_INDISPONIBLE` : un article est épuisé ou supprimé ;

— `POINTS_INSUFFISANTS` : le solde est inférieur aux points demandés ;

— `POINTS_INVALIDES` : les points ne sont pas un multiple de 15, ou la réduction dépasse le sous-total ;

— `HORS_HORAIRES` : l'heure d'arrivée est hors des horaires d'ouverture ;

— `VALIDATION_ERROR` : champ manquant ou incohérent avec le mode de service.

Un jeton absent ou expiré donne `401`.

6 Lien avec les spécifications

Diagramme	Parties du dossier de spécifications	Endpoints
Cas d'utilisation	Récapitulatif des endpoints ; section 7	Les 15 endpoints du module
Classes	Règles de gestion (section 4) ; réponses JSON de la section 7	Champs des réponses des endpoints
Séquence : inscription	Section 7.1.1	`POST /api/auth/register`
Séquence : connexion	Section 7.1.2	`POST /api/auth/login`
Séquence : commande	Section 7.3.1 ; règles de calcul des points (section 4)	`POST /api/orders`

7 Points à valider avec le groupe

— **Caleb (base de données) :** les noms et les types des attributs du diagramme de classes doivent correspondre à son schéma.

— **Aksa (back-office) :** les attributs `traite_par` et `valide_par` de `Reclamation` ; les statuts de commande et de réclamation, qui servent de référence commune.

— **Yviline (fidélité et animation) :** l'ajout des classes de parrainage, de promotion et d'événement lors de la fusion des diagrammes ; l'éventuel type de mouvement `parrainage` ; le champ `code_parrainage` à l'inscription.

— **Module de paiement (bonus) :** les valeurs `paye` et `echoue` de `StatutPaiement`, proposées ici, à confirmer.